

Rapport de projet et d'étude

NFE204 : bases de données documentaires et distribuées

Benoît THIERRY, auditeur. – siscot 100110030

Premier semestre 2014-2015

v1 rendue le 28/02/2015

v1.1 du 02/03/2015

SUJET. DANS LE CADRE DU COURS « BASES DE DONNÉES DOCUMENTAIRES ET DISTRIBUÉES », NOUS AVONS RÉALISÉ UN « MINI-PROJET » QUI COMPRENAIT DEUX PARTIES :

- PRATIQUE/UTILISATION : CONSTITUER UNE BASE DE DONNÉES DOCUMENTAIRE AVEC UN DES NOMBREUX SYSTÈMES NOSQL DISPONIBLES ;

- ETUDE : APPROFONDIR UN DES ASPECTS DU SYSTÈME NOSQL CHOISI.

RESUME. NOUS AVONS D'ABORD CONSTITUÉ UNE BASE D'OBSERVATIONS GÉOLOGIQUE A PARTIR DE DONNÉES DISPONIBLE EN WEB-SERVICES SUR INTERNET. CES DONNÉES ONT ÉTÉ COMPILÉES AVEC MONGODB, PUIS INDEXÉES AVEC 2DSphere ET ELASTICSEARCH.

QUAND À L'ÉTUDE, ELLE EST SÉPARÉE EN 2 PARTIES AFIN DE RÉPONDRE AUX BESOINS SPÉCIFIQUES LIÉS À L'ANALYSE DE CES DONNÉES GÉOLOGIQUES. NOUS AVONS D'ABORD RECHERCHÉ LES MEILLEURS MOYENS D'INDEXATION DE DONNÉES SPATIO-TEMPORELLES (3 OU 4 DIMENSIONS), PUIS NOUS AVONS ENSUITE ÉTUDIÉ LA POSSIBILITÉ D'EXPLOITER UNE BASE NOSQL HYBRIDE, MÉLANGEANT LE PARADIGME DES GRAPHS AVEC CELUI DES DOCUMENTS SEMI-STRUCTURÉS. NOS EXPLORATIONS NOUS ONT NOTAMMENT AMENÉS VERS LE SYSTÈME ORIENTDB.

le cnam

CONSERVATOIRE NATIONAL
DES ARTS ET MÉTIERS
2, RUE CONTÉ
75003 PARIS

Table des matières

Introduction.....	4
I.1.Objet de ce document.....	4
I.2.Structure de ce document.....	4
Expression des besoins.....	5
II.1.Description générale.....	5
II.1.1.Domaine fonctionnel.....	5
II.1.2.Objectifs généraux.....	5
II.2.Besoins en collecte de données.....	6
II.2.1.Sources de données choisies.....	6
II.2.2.Besoins en persistance des données.....	6
II.3.Description prévisionnelle de problématiques d'analyse.....	7
II.3.1.Navigation et représentation graphique.....	7
II.3.2.Fouille de données.....	7
II.4.Identification des besoins en manipulation de données.....	8
II.4.1.Requêtes temporelles.....	8
II.4.2.Requêtes spatiales.....	8
II.4.3.Requêtes en langage symbolique.....	8
II.4.4.Autres fonctionnalités géospatiales utiles.....	8
II.5.Identification des besoins en modélisation des données.....	9
II.6.Identification des besoins en stockage.....	9
Mise en œuvre.....	11
III.1.Démarche de travail.....	11
III.2.Collecte des données.....	11
III.2.1.Format et dimensionnement.....	11
III.3.Adéquations des outils essayés avec les besoins exprimés.....	15
III.3.1.Solutions en exploitation de données spatio-temporelle.....	15
III.3.2.Interrogation des données.....	15
III.3.3.Autres fonctionnalités géospatiales utiles.....	16
III.4.Faiblesse des bases relationnelles avec des données spatio-temporelles.....	17
III.5.Rappel : indexation classique de données par arbre-B+.....	17
III.6.Première évolution : Calcul de clef d'index par hachage de trajectoire spatiale.....	18
III.7. Pour plus de précision : indexation par arbre R.....	19
III.8.Solution alternative : indexation des geohash avec Lucene/ElasticSearch.....	20
Pour aller plus loin : vers un modèle de données hybride.....	22
IV.1.1.Rappel des besoins.....	22
IV.1.2.Rappel des principaux paradigmes de modélisation de données.....	23
IV.1.3.Et maintenant : pourquoi se limiter à un seul paradigme ?.....	24
IV.2.Manipulation de données orientées graphes : quels langages ?.....	26
IV.2.1.1.Le langage SPARQL.....	26
IV.3.Solution 1 : AllegroGraph.....	26
IV.4.Solution 2 : OrientDB.....	27
IV.4.1.Caractéristiques retenues.....	27

Index des figures

Figure 1. Diagramme de classes du modèle de données QuakeML (source A).	12
Figure 2. Exemple d'arbre B+.....	17
Figure 3. Trajectoire spatiale en Z sur une planisphère.....	18
Figure 4. Exemple de segmentation de plan par arbre-R.....	19
Figure 5. Exemple de segmentation tridimensionnelle par arbre-R.....	20
Figure 6. Les principaux paradigmes de modélisation de données.....	23

Index des tableaux

Tableau 1. Principaux outils à modélisation hybride disponibles actuellement.....	25
--	----

Partie I

Introduction

I.1. Objet de ce document

Ce document fait le rapport de l'étude, à la fois théorique et pratique, réalisée pour le « mini-projet » (sic) personnel portant sur les sujets vus au en cours de « bases de données documentaires et distribuées » (réf. CNAM **NFE204**).

Ce document s'inscrit également dans le déroulement prévu du cursus pour l'obtention du certificat « Analyste de données massives ».

I.2. Structure de ce document

Ainsi que demandé dans le document [projet.html](#), le projet décrit dans ce document comprend deux aspects :

- Pratique/utilisation : constituer une base de données documentaire avec un des nombreux systèmes NoSQL disponibles
- Étude : approfondir un des aspects du système NoSQL choisi.

« Ce schéma [pouvant] accepter diverses variations », nous avons pour la seconde partie étudiés 2 aspects distincts des solutions NoSQL :

- d'abord, l'indexation de données spatio-temporelles (à distinguer de la seule indexation géo-spatiale) avec les principales solutions théoriques et outils disponibles librement ;
- ensuite, les outils permettant la modélisation et la manipulation de données en combinant plusieurs paradigmes dans un même entrepôt : ce que nous appelons « modèles de données hybrides ».

Ces deux aspects font chacun partie du projet décrit en **partie II : Expression des besoins**. Cependant, comme le premier aspect a pu être immédiatement mis en pratique, il est décrit en **partie III : Mise en œuvre**.

A l'opposé, le second aspect, plus complexe, se limite pour le moment à une prospective sur une évolution utile à la suite de ce projet personnel, c'est à dire pour la fouille et l'analyse de données . C'est pourquoi aucune réelle mise en application n'a été décrite, et qu'on retrouve cette étude dans une partie séparée de ce document : **Partie IV : Pour aller plus loin**.

Partie II

Expression des besoins

Cette partie du rapport rappelle d'abord les différentes motivations à l'origine de ce projet, puis fait le détail des besoins fonctionnels et techniques nécessaires à sa réalisation.

II.1. Description générale

II.1.1. *Domaine fonctionnel*

Ce projet traite de l'archivage et la compilation de données géologiques, d'abord sous forme de mesures physiques, mais aussi sociales (conséquences factuelles connues sur les populations). L'ensemble de la collecte se fait à l'échelle de l'histoire humaine (une partie basée sur des mesures temps-réel, une autre partie en *paléogéologie*).

II.1.2. *Objectifs généraux*

Ce projet est réalisé hors temps de travail et son objectif est essentiellement pédagogique. Nous voulons avant tout préparer des données d'entrées destinées à alimenter des algorithmes de classification et de prévision, ceci dans un volume suffisamment conséquent pour justifier une approche « données massives » et une distribution des traitements.

Le choix du domaine des données géologiques et sociales nous a permis de structurer une base de connaissances qui doit nous permettre d'effectuer des observations et des liens de cause à effet au sujet de différentes catégories de phénomènes géologiques, soit par traitement automatisé, soit par analyse.

Ce document traite donc exclusivement des aspects collecte des données et mise en place d'un système de stockage, de manipulation et de traitement distribué.

La seconde facette de ce projet personnel au CNAM consistera à chercher à exploiter ces mêmes données de façon automatisée afin de produire un panel d'observations ou de prédictions géologiques. Les techniques employées pour ce travail ne se baseront pas sur des expertises de la science géologique, mais au contraire sur une méthodologie générale d'analyse et de déduction.

Le développement ultérieur de ce sujet pour les cours de **RCP216** abordera les possibilités d'exploration et d'extraction de connaissances sur ce *corpus*. Une dernière partie, dédiée au cours de **STA211**, abordera, quand à elle, une approche plus analytique sur l'interprétation des résultats.

II.2. Besoins en collecte de données

II.2.1. Sources de données choisies

Il y a apparemment différents domaines de l'activité humaine produisant des données relatives aux phénomènes géologiques : d'abord les organismes scientifiques, mais aussi les études sociales gouvernementales concernant la gestion de risque écologique ou sismique. Nous avons relevés plusieurs sources potentielles :

- Le site international *Seismic Portal* (<http://www.seismicportal.eu/jetspeed/portal/>) fournit apparemment des mesures concernant les événements sismiques survenus depuis l'année 1999 (source **A**),
- L'américain *Earthquake Database* (<http://www.ngdc.noaa.gov/ngdc.html>) répertorie les tremblements de terre connus ayant engendré des destructions significatives entre 2150 avant J.-C. et aujourd'hui. Il s'agit donc majoritairement de données *paléogéologiques* (source **B**),
- Les relevés de mesures sismique **en temps réel** fournies par différents organismes nationaux comme l'EMSC (<http://www.emsc-csem.org>) (source **C**) pourraient nourrir un modèle de prédiction (la plage temporelle des données étudiées en temps réel s'étalant tout de même sur une dizaine d'années).
- En complément, il pourrait être possible dans un second temps d'exploiter des données hydrographiques pour évaluer l'*aléa sismique* lié aux chocs hydrologiques locaux (pour les USA : <http://nhd.usgs.gov/data.html>). Au cas où ces données hydrométriques manqueraient sur la zone et l'époque étudiée, des données pluviométriques pourraient éventuellement les remplacer (<http://openweathermap.org/api>). (source **D**).

II.2.2. Besoins en persistance des données

Comme évoqué plus haut, on souhaite d'abord compiler des données géologiques et sociales de plusieurs origines pour réussir dans un second temps à structurer une base de connaissances sur les événements marquants. Il y a donc en fait deux besoins de stockage distincts :

- besoin n°1 : d'abord une base de données brutes, sous forme de données de mesures ou d'observations. Seuls le flux « temps-réel » (source **C**) demandera des écritures régulières. Elles resteront cependant occasionnelles (une fois par jour environ) mais pourrons ce produire à n'importe quel moment pendant l'activité du système. Dans ce cadre, une base NoSQL favorisant la cohérence des données plutôt que la haute disponibilité en écriture sera parfaite. **MongoDB** est donc un candidat très valable sur ce besoin.
- besoin n°2 : dans le cadre de la poursuite de ce projet en fouille de données, nous aurons également besoin de construire une base de faits à partir des données brutes. Cette base de faits doit permettre d'enregistrer les liaisons ou relations calculées entre les différentes observations (par exemple : la proximité géographique entre 2 centre d'activité sismique, associés également par la distance temporelle des différentes manifestations et la nature des manifestations physiques). Il s'agit de données en graphe. L'archivage de ce type de données est reconnu peu efficace dans un SGBD relationnel, mais aussi dans un système à base de collections comme **MongoDB**, car ici la majorité des données sont des relations entre les enregistrements, et non des valeurs littérales. Les relations étant dans ces 2 cas stockées sous forme de références d'enregistrements, les procédures de recherche prennent énormément de temps car il

faut plusieurs parcours de la même table ou de la même collection pour obtenir l'ensemble des nœuds d'un graphe. Les « jointures » multiples de table deviennent coûteuses. Quand à **MongoDB** en particulier, il est incapable de réaliser lui-même l'équivalent de la jointure relationnelle. C'est une des raisons pour lesquelles il existe des SGBD dédiés aux structures de graphes qui remplacent simplement le processus de jointure de collections indexées par des références directes des objets entre eux. Les bases NoSQL les plus connues dans ce contexte sont **Neo4j** et **OrientDB**. Nous avons finalement retenu **OrientDB**, disponible partiellement sous licence Apache. Nous justifions principalement ce choix par la plus grande souplesse du modèle de données, bien que sa conception originale lui fournisse d'autres avantages. Consulter à ce sujet la dernière partie de ce document (partie IV *Pour aller plus loin*).

II.3. Description prévisionnelle de problématiques d'analyse

II.3.1. Navigation et représentation graphique

La première fonction envisagée doit permettre de naviguer dans les données recueillies selon des critères spatio-temporels, c'est à dire à l'aide d'un moteur de recherche :

- En fonction de la date de l'observation ou, au contraire, en cumulant les mesures sur une période temporelle,
- En fonction d'une zone géographique spécifique.

Les résultats seront d'abord disponibles sous forme de *listing*.

Une représentation graphique des mesures compilées pourra ensuite avoir lieu sur une planisphère.

D'autres représentations graphiques de type statistiques pourront ensuite être ajoutées pour les besoins de l'analyse, à l'aide de produits dédiés.

II.3.2. Fouille de données

Il est difficile d'évaluer aujourd'hui ce qu'il sera possible de réaliser dans le cadre de ce projet hors temps de travail, ceci à partir des données qui vont être récoltées et des moyens théoriques et techniques fournis ultérieurement dans le cadre des cours **RCP216** et **STA211**.

Voici déjà cependant quelques idées, non confirmées pour le moment :

- Identification automatique de phénomènes géologiques vérifiables comme les principales failles inter-continentales et intra-continentales, les corrélations avec les anomalies sismiques, ou encore les plus grands foyers hydrogéologiques disponibles selon les saisons, ...
- Identification automatique et spatio-temporelle des grandes catastrophes sismiques de l'histoire et de la paléo-histoire, toujours à partir des données brutes,
- Tout à fait idéalement, un modèle de prévision de l'*aléa sismique*, local ou global. A noter qu'il n'existe à ce jour (2014-2015) aucun modèle réputé totalement fiable sur ce sujet.

II.4. Identification des besoins en manipulation de données

Les données sont de nature spatio-temporelles en 4 dimensions (latitude, longitude, altitude et temps) et non juste géo-spatiales. Les besoins sont donc un peu plus pointus.

II.4.1. Requêtes temporelles

- requête de proximité temporelle par rapport à un événement géologique spécifié

II.4.2. Requêtes spatiales

- requêtes de portée/proximité/voisinage par rapport à un artefact géologique ou un événement géologique: permettra de retrouver rapidement les antécédents sismique d'une région
- requêtes d'appartenance à une zone géographique de type surface géo-localisée : polygone, surface circulaire, intersections, ... servira par exemple à localiser les zones de à risque sismique le long d'un cours d'eau.
- requêtes des k-plus proches voisins (« k-ppv »), essentielle pour les classifications

II.4.3. Requêtes en langage symbolique

- requêtes topologiques. Exemple : quels sont les massifs montagneux à proximité d'une ville de plus de 1000 habitants ?

II.4.4. Autres fonctionnalités géospatiales utiles

En plus des simples interrogations de données, un certain nombre de fonctionnalités spécifiques est couramment proposé pour interagir facilement avec des données géo-spatiales. Voici la liste spécifiée par le standard « Simple Features » de l'*Open Geospatial Consortium* :

- mesures dans l'espace : calculer des longueurs de ligne, des surfaces polygonales, des distances entre différents types de géométrie.
- opérateurs spatiaux : possibilité de modifier des géométries existantes ou créer de nouvelles géométries à partir des existantes, par composition ou suppression (exemples : ajouter un périmètre autour d'un objet, calculer une intersection)
- constructeurs géométriques : ici on doit posséder des outils pour insérer dans la base une géométrie à partir de nouvelles données fournies de manière transactionnelle (équivalent du INSERT en SQL)
- prédicats spatiaux : capacité à poser des questions d'ordre topologiques (déjà mentionné dans le point précédent). Exemple : recouvrement ou non de 2 polygones, combien d'habitations situées à proximité d'un chantier de construction...
- fonctions d'observation : capacité à obtenir des informations géométriques sur les éléments visibles depuis un point de vue. Exemple : centre d'un cercle délimité par le champ de vision.

Ces standard s'adresse essentiellement aux bases de type GIS, mais certains outils NoSQL implémentent en partie cette spécification.

II.5. Identification des besoins en modélisation des données

- peu de contraintes d'intégrité : Chacune des différentes source de données identifiées possède un schéma assez précis mais beaucoup de champ peuvent être vide.
Dans le format QuakeML, peu de champs ont une présence garantie (hormis les identifiants). La possibilité de se passer de schéma de données nous permettrait de travailler sur plusieurs sources de données suivant ce standard, indépendamment du fait que les champs facultatifs puissent être plus ou moins renseignés selon l'origine et la période.
D'autre part, vu l'objectif expérimental et exploratoire de notre entrepôt, on veut également pouvoir intégrer de nouvelles sources de données au fur et à mesure de leur découverte. C'est pourquoi on veut enrichir rapidement et simplement le modèle de données sans avoir à réécrire le schéma et la couche d'accès aux données à chaque fois.
- peu de valeurs continues : Les valeurs de la majorité des champs sont discrètes (nombre fini de valeurs) car les valeurs continues peuvent facilement être ramenées à des plages
- pas de champs multivalués : A priori, pas besoin de valeurs de champs de type collection
- modèle hiérarchique : un modèle de données avec d'enregistrements imbriqués nous permettra de garder les enregistrements structurés sans recourir à un renommage expansif des noms d'attributs.
- volume important de relations : On a besoin d'insérer régulièrement de nouvelles relations entre éléments, selon la sémantique visée (exemple : les relations « se trouve à proximité géographique de » sont paramétrables selon les dimensions de l'aire géographique étudiée et doivent donc être recalculés préalablement à l'étude).
- relations sans limites de schéma : On doit pouvoir interconnecter les éléments par un nombre indéterminé de relations. En effet, pour utiliser des méthodes exploratoires de fouille de données dans la recherche de motifs, on a besoin de recueillir rapidement les proximités entre les éléments. Ces proximités sont autant de relations qui sont amenées à être modifiées.

II.6. Identification des besoins en stockage

- Indexation multi-dimensionnelle : Les motifs de données événementielles (Source A, par exemple) n'ont pas de position prédéterminée dans le temps et l'espace, on a donc besoin d'une indexation à la fois géospatiale et temporelle.
- Indexes secondaires : La meilleure façon de faire fonctionner rapidement plusieurs types de requêtes différentes est d'utiliser plusieurs indexes sur les mêmes données. Tous les SGBD ne le permettent pas.
- MapReduce et agrégations : De part la présence de nombreuses données chiffrées, on doit pouvoir produire des agrégations de ces champs. MapReduce est l'outil idéal pour distribuer ce type de traitements.
- Agrégations simple : indépendamment de MapReduce qui demande un travail de contrôle important, il est également possible de chercher un moteur d'agrégation spécialisé qui fournisse de meilleures performances.
- Partitionnement : Chaque échantillon doit être suffisamment important pour ne pas trop être sensible au bruit : nous avons donc besoin d'analyser conjointement un volume

important de données. Une capacité de partitionnement des données est donc indispensable.

- Scalabilité : de par les volumes alloués, on souhaite une capacité de montée en puissance proportionnelle aux ressources allouées (même si la linéarité n'est pas indispensable).
- Élasticité peu utile : de part la nature relativement prévisible du volume de données (consulter II.2.1 Sources de données choisies) à ajouter au fil du temps, l'élasticité des capacités de stockage et de calcul ne semble donc pas être une priorité sur ce projet, au contraire de la scalabilité dont nous venons de parler.

Partie III

Mise en œuvre

Cette partie du rapport a pour but de faire la correspondance entre les besoins exprimés en Partie II et les outils théoriques et techniques disponibles pour la réalisation de ce projet personnel.

III.1. Démarche de travail

Nous n'avons pas pour prétention de réaliser dans le cadre de ce document un état de l'art exhaustif de la technique ou du marché du logiciel NoSQL, c'est pourquoi nous nous concentrons d'emblée sur les logiciels libres d'utilisation dans un cadre non-lucratif. Dans ce cadre déjà restreint, et pour chacun des besoins exprimés nous énumérerons les principales méthodes théoriques. A cet aspect théorique, nous opposerons ensuite la réalité des outils disponibles librement.

La fin de ce chapitre nous permettra de résumer sur le choix des outils actuellement utilisés pour le projet d'analyse de données au CNAM et sur les perspectives d'évolution ou de complément par d'autres outils.

III.2. Collecte des données

III.2.1. *Format et dimensionnement*

Note : Consulter la partie II Expression des besoins plus haut pour retrouver l'origine et l'aspect fonctionnel de ces 4 sources de données.

- Source A : suit le schéma de la norme QuakeML, format spécifique de description sismique, exportable aux formats JSON ou XML. La source produit apparemment environ 600 enregistrements par semaine, le volume global sur 14 ans d'activité est donc environ de $600 \times 52 \times 14 \approx 438.300$ enregistrements récupérables aujourd'hui Le service est bridé à 1000 enregistrements par appel. : il nous a donc fallu récupérer ces données par tronçons et, dans un premier temps, selon les périodes temporelles visées par l'analyse.

Le schéma complet de QuakeML est fournit ici en figure 1. Cependant, on a constaté que plusieurs des objets ne sont pas fournis dans les données de la source A, et que certains gens ne sont pas nécessairement renseignés.

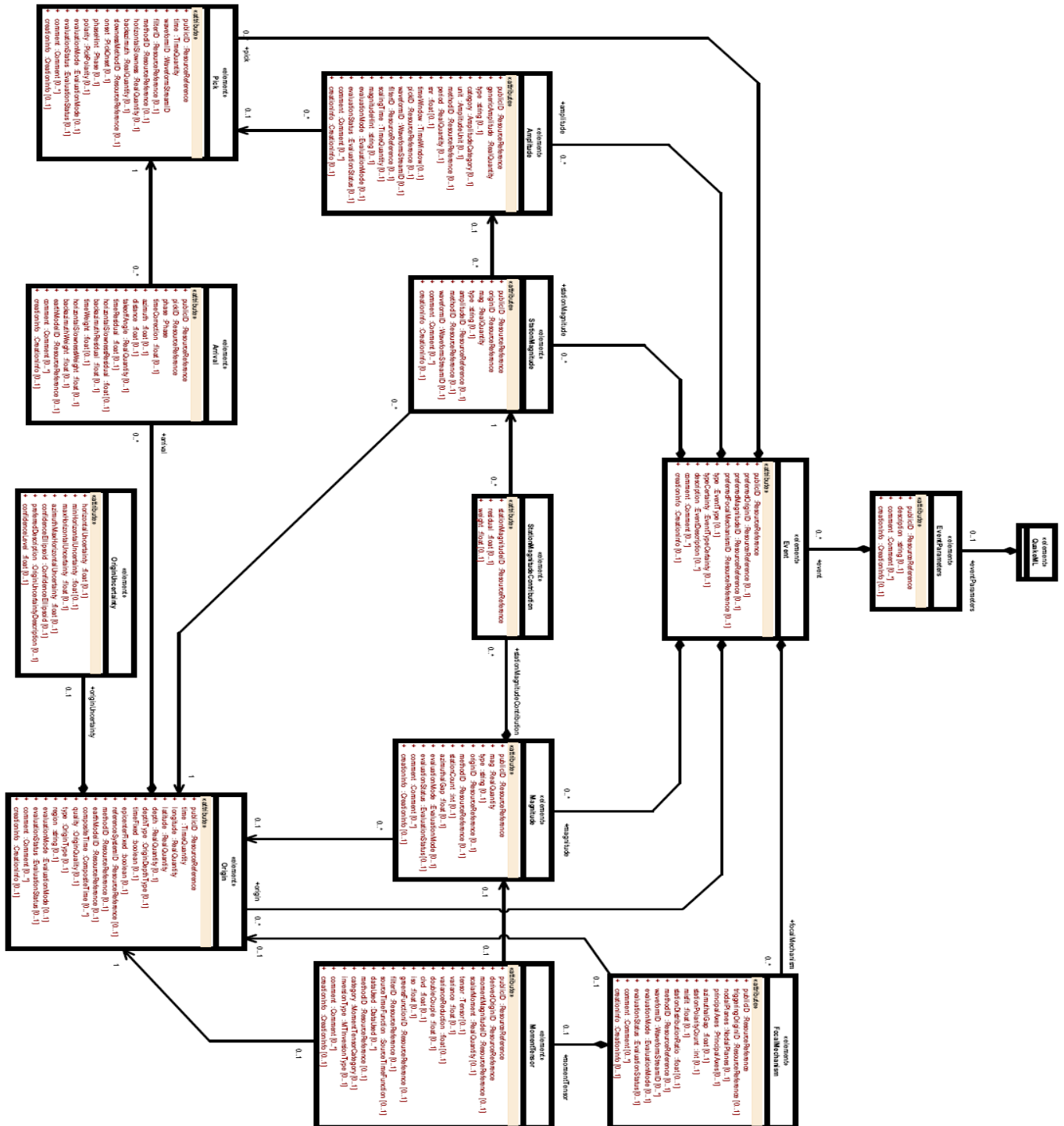


Figure 1. Diagramme de classes du modèle de données QuakeML (source A)

La collecte des données a été faite en interrogeant un service web à l'aide du script suivant qui est paramétrable sur la plage calendaires des événements sismique recueillis. Nous n'avons pour le moment pas eu besoin d'en automatiser l'appel car le volume de données entre 2009 et 2015 est déjà suffisamment considérable pour servir de base à des analyses.

```
#!/bin/bash

set -e # tte erreur est fatale

# paramètres
let date_start='20090101'
let date_end='20141231'

let date_end=$(date +%Y%m%d -d "$(date "+%Y%m%d" -d ${date_end}) + 1 day")
echo $date_end

let limit=1000

format='json'

rootpath='data2011'

let YEAR=$(date +%Y)
let MONTH=$(date +%m)
let DAY=$(date +%d)

counter=1
let nb_records=1000
starttime=`date +%Y%m%d -d ${date_start}`
#endtime=`date +%Y%m%d -d "${date_start} + 1 month"`
endtime=`date +%Y%m%d -d "${date_start} + 1 month"`
echo $starttime
echo $endtime
echo $date_start

while [ $starttime -lt "${date_end}" ]; do

# JSON
wget \
--page-requisites \
--restrict-file-names=windows \
--no-parent \
--no-directories \
--directory-prefix="${rootpath}" \
--adjust-extension \
--output-document="${rootpath}/start=$(date +%Y-%m-%d -d "${starttime}")&end=$(date +%Y-%m-%d -d "${endtime}").json" \
--no-clobber \
--no-verbose \
--show-progress \
"$@" \
"http://www.seismicportal.eu/fdsnws/event/1/query?limit=$limit&start=$(date +%Y-%m-%d -d ${starttime})&end=$(date +%Y-%m-%d -d ${endtime})&format=json"

# XML
```

```
wget \
--page-requisites \
--restrict-file-names=windows \
--no-parent \
--no-directories \
--directory-prefix="${rootpath}" \
--adjust-extension \
--output-document="${rootpath}\start=$(date +%Y-%m-%d -d "${starttime}")&end=$(date +%Y-%m-%d -d "${endtime}")&end=$(date +%Y-%m-%d -d "${endtime}")&format=xml" \
--no-clobber \
--no-verbose \
--show-progress \
"$@" \
"http://www.seismicportal.eu/fdsnws/event/1/query?limit=$limit&start=$(date +%Y-%m-%d -d "${starttime}")&end=$(date +%Y-%m-%d -d "${endtime}")&format=xml"

starttime=`date +%Y%m%d -d "$starttime + 1 month"`
endtime=`date +%Y%m%d -d "$starttime + 1 month"`
counter=$(( $counter + 1 ))
done

echo 'Collecte achevée.'

exit $?
```

Nous avons ensuite importé les données JSON dans **MongoDB** à l'aide de **mongoimport**. Mais le format n'est pas directement compatible car :chaque fichier comporte un entête et un marqueur de fin de fichier, que **mongoimport** est incapable d'éviter. Nous avons donc écrit un script supplémentaire importer en une seule fois la totalité des fichiers json disponibles et reformater chacun de ces fichiers en remplaçant l'entete par une structure de collection JSON.

```
#!/bin/bash

set -e # tte erreur est fatale

#paramètres généraux
rootpath='data2011'

available_files=$(ls "${rootpath}/*.json")

for input_file in ${available_files[@]}
do
    echo $input_file
    { echo '['; tail -n +2 $input_file | head -c -1; } | mongoimport
--db geo --collection events --jsonArray --upsert
done
```

- Source **B** : environ 6000 enregistrements au format CSV, récupérées en une fois (données d'archive). Nous prévoyons de le mettre à jour annuellement.
- Source **C** : données temps réel à volume variable. Après vérification, le format semble finalement assez difficile à exploiter rapidement, car il s'agit d'un format binaire spécifique qui paraît complexe. Bien qu'il existe apparemment plusieurs outils amateur

ayant pour but de convertir ce format vers des formats compatibles avec des outils d'analyse comme Matlab, nous avons écarté cette source pour le moment.

- Source **D** : Les données hydrographiques sont disponibles dans des formats différents selon les pays et organismes fournisseurs. Dans le cas des USA, il s'agit de tables au format .DBF qu'il faudra convertir en JSON. Le volume exact est difficile à déterminer, car il dépend de la zone utilisée (par exemple : les états-unis sont fournis individuellement par état). Pour la raison qu'il est impossible d'automatiser cette source de données, nous l'avons écartée pour le moment.

III.3. Adéquations des outils essayés avec les besoins exprimés

Nous avons, pour le moment, utilisé **MongoDB** et **Elasticsearch** pour réaliser une solution répondant au besoin n°1 décrit en partie II Expression des besoins.. Nous donnons ici les réponses identifiées aux différents pré-requis techniques ou fonctionnels détaillés dans cette partie.

III.3.1. Solutions en exploitation de données spatio-temporelle

Un index **2DSphere** a été créé dans MongoDB sur la collection d'observations de la source **A**. Un des avantages de **2DSphere** par rapport à l'index **2D** est qu'il offre un ensemble d'opérateurs spécialisés pour les données géolocalisées.

Cependant, les localisations de chaque observation sismique comportent 3 coordonnées (latitude, longitude, altitude) ; hors, **2DSphere** refuse d'accepter plus de 2 coordonnées depuis la version 2.4 de **MongoDB**. Nous avons donc écrit une requête spécifique pour supprimer la représentation de l'altitude de chaque élément de la collection, cette donnée étant déjà disponible dans chaque enregistrement sous un champs séparé.

```
db.events.update({}, { $unset: {'geometry.coordinates.2' : 1}}, {multi : true})
db.events.update({}, { $pull : {"geometry.coordinates" : null}}, {multi : true})
```

Grâce à cette modification, nous pouvons ensuite la création de l'index :

```
db.events.ensureIndex( { type : 1, geometry : "2dsphere",
"properties.time" : 1 } )
```

III.3.2. Interrogation des données

- requêtes de portée/proximité/voisinage : aggrégation GEONEAR

```
db.runCommand( {
  geoNear: <collection> ,
  near: { type: "Point" , coordinates: [ <coordinates> ] } ,
  spherical: true,
  ...
} )
```

- requêtes d'appartenance à une zone géographique : **2DSphere** est capable d'exprimer des requêtes de pattern-matching sur tous types d'objets vectoriels au format GEOJSON grâce à la commande \$GEOWITHIN.

```
db.places.find(
{
  loc: {
    $geoWithin: {
```

```

    $geometry: {
      type: "Polygon",
      coordinates: [ [ [ 0, 0 ], [ 3, 6 ], [ 6, 1 ], [ 0, 0 ] ] ]
    }
  }
}
)

```

- requêtes des k-plus proches voisins (« k-ppv ») : la commande `GEONEAR` vue plus haut fournit comme résultat les éléments les plus proches avec un ordre de distance. Il est donc facile de sélectionner les k plus proches.
- requêtes topologiques : **2DSphere** ne supporte que 2 opérateurs topologiques : `WITHIN` et `INTERSECTS`. Ce qui est plutôt limité : il n'y a rien de prévu comme fonctions d'analyse (zone de « tampon » autour d'une géométrie, enveloppe convexe, ...) ou ensemblistes (union, différence...). Les recherches de ce type devront donc se faire à l'aide d'API séparées, idéalement utilisables depuis des traitements MapReduce, pour un traitement distribué.

III.3.3. Autres fonctionnalités géospatiales utiles

Nous avons détaillé en II.4.4 Autres fonctionnalités géospatiales utiles les fonctionnalités géospatiales spécifiées par le standard « Simple Features » de l'*Open Geospatial Consortium*.

Nous indiquons ici les solutions utilisables avec **MongoDB**.

- mesures dans l'espace : l'opérateur `$GEO_NEAR` dont nous avons déjà parlé est également capable de retourner dans le résultat les calculs de distance qui lui ont permis de déterminer un voisinage.

```

db.collection.aggregate([{$geoNear:{near:
[LON,LAT],distanceField:"distance",maxDistance:RADIUS,spherical:true}}])

```

- opérateurs spatiaux : de la même façon que pour les requêtes topologiques, il n'existe pas dans MongoDB d'opérateurs ensemblistes permettant de créer une géométrie à partir d'une autre.
- constructeurs géométriques : il est possible d'ajouter n'importe quel élément à la collection par un simple `add()`, du moment que l'on connaît la valeur de tous ses champs.
- prédicats spatiaux : **2DSphere** ne supporte que 2 opérateurs topologiques : `WITHIN` et `INTERSECTS`. Ce qui est très limité vu qu'il n'y a rien de prévu pour des fonctions d'analyse (zone de « tampon » autour d'une localisation, enveloppe convexe, ...) ou ensemblistes (union, différence...).
- fonctions d'observation : il n'y a pas de fonctions géométriques pour travailler sur les résultats, mais en revanche, la majorité des fonctions de **2DSphere** acceptent en paramètre ou retournent comme résultat des objets GEOJSON, un format capable de décrire un peu près n'importe quel type de géométrie. Il manque tout de même quelques éléments importants comme le type Cercle, qui ne peut pour le moment être représenté que comme un polygone.

```

{ "type": "Feature",
  "geometry": {
    "type": "Polygon",
    "coordinates": [
      [ [100.0, 0.0], [101.0, 0.0], [101.0, 1.0],
        [100.0, 1.0], [100.0, 0.0] ]
    ]
  }
}

```


}

En conclusion, l'usage nous a donc démontré que **MongoDB** ne remplit qu'imparfaitement les fonctions géospatiales attendues d'un GIS. Ce manque risque d'être problématique pour le projet en cas d'analyse poussée sur des régions géographiques décrites précisément (vectorielles). Il faudra alors compléter avec des outils géométriques externes.

III.4. Faiblesse des bases relationnelles avec des données spatio-temporelles

les bases spatio-temporelles ne sont généralement pas basées sur le modèle relationnel pour plusieurs raisons :

- les entités ne se limitent typiquement pas à une position géographique, mais à une plage de positions (exemples : étendue non-uniforme d'un séisme, coordonnées de l'ensemble d'une chaîne de montagne comme site géologique...), ce qui oblige à parcourir les tables pour faire des sélections de valeurs par inégalités.
- ces données sont multidimensionnelles par définition (4 dimensions, dont le temps), ce qui implique pour chaque requête sur plage du modèle conceptuel soit en réalité implémentée par 4 sous-jointures. Dans le cas de recherche par plage de coordonnées (proximités, k-ppv, ...), ce sont autant de conditions de jointures par inégalité combinées pour un seul résultat : font partie des jointures les plus coûteuses en temps.
- si les entités doivent pouvoir se déplacer dans l'espace ou le temps, il faut systématiquement remettre à jour l'ensemble des indexes de type arbre B+ : ce qui est coûteux car il en faut au minimum $3 + 1 + 1 = 5$ (longitude, latitude, altitude, temps et le composite (longitude+latitude+altitude+temps))

Dans la pratique, certains systèmes relationnels sont vendus avec des extensions spécifiques destinées aux GIS, ces extensions comprennent précisément un type spécifique d'index, dit « par arbre R ». C'est le cas de **Oracle Spatial**.

III.5. Rappel : indexation classique de données par arbre-B+

Tous les SGBDR du marché fondés sur le langage SQL semblent utiliser cette structure pour réaliser les indexes sur des clef composites. Les systèmes NoSQL généraux (c'est à dire non dédiés à un type d'usage particulier) proposent essentiellement ce type d'index. Nous allons voir que ce type d'indexation n'est pourtant pas optimal pour tous les types de clefs.

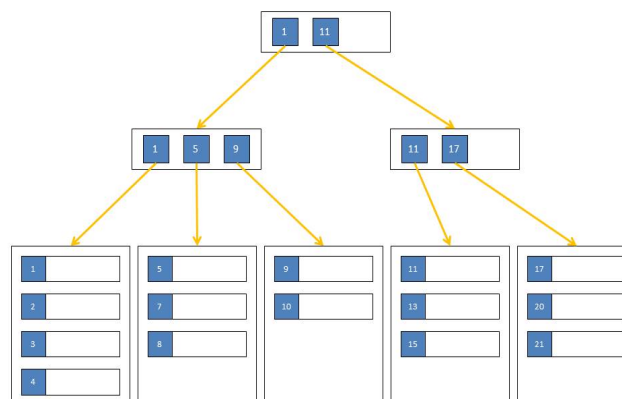


Figure 2. Exemple d'arbre B+

L'arbre B+ est une structure hiérarchique des clef qui est réajustée à chaque insertion. Les branches de l'arbre sont rééquilibrées lorsque qu'un seuil est dépassé. La particularité du B+ par rapport à d'autres structures d'arbres jugées moins efficaces est que toute les « feuilles » se trouvent à tout moment à même distance de la racine, ceci permettant d'avoir un temps d'accès presque identique pour n'importe quelle donnée. Son autre particularité est que seules les « feuilles » contiennent les données de la base, ce qui permet d'effectuer la majorité des modifications ou insertions en accès séquentiel (par bloc), avec les bénéfices connus en matière de performances.

La complexité en opérations d'une recherche individuelle est de l'ordre $O(\log n)$, n étant le volume des données. Celle pour une insertion est de l'ordre $O(n)$.

Mais cette organisation est rapidement inefficace lorsque la recherche se fait simultanément sur plus d'une dimension comme c'est le cas pour les indexes spatio-temporel : il n'existe alors aucun ordre de tri pour la clef composite qui maximise les performances de recherche par plages sur plusieurs dimensions. Plus le nombre de dimensions est important (dans notre cas, 4 dimensions), plus le problème se fait sentir.

III.6. Première évolution : Calcul de clef d'index par hachage de trajectoire spatiale

L'algorithme de ce type le plus connu est celui mis à disposition publiquement sous le nom de QUADTREE et GEOHASH (2008). QUADTREE ET GEOHASH utilisent une trajectoire en Z pour parcourir un plan selon des zones rectangulaires sans recouvrements.

Pour QUADTREE, le principe consiste à diviser le plan en quartets portant chacun une valeur de clef de 1 à 4, puis à subdiviser chacun d'entre eux de la même façon, et ainsi de suite. En utilisant un encodage binaire, et en concaténant les sous-clefs entre elles, on obtient un identifiant unique de toute zone du globe terrestre, avec un choix libre de la granularité, selon le nombre de sous-niveaux retenu. La longueur de la chaîne binaire servant de clef est fonction de cette granularité.

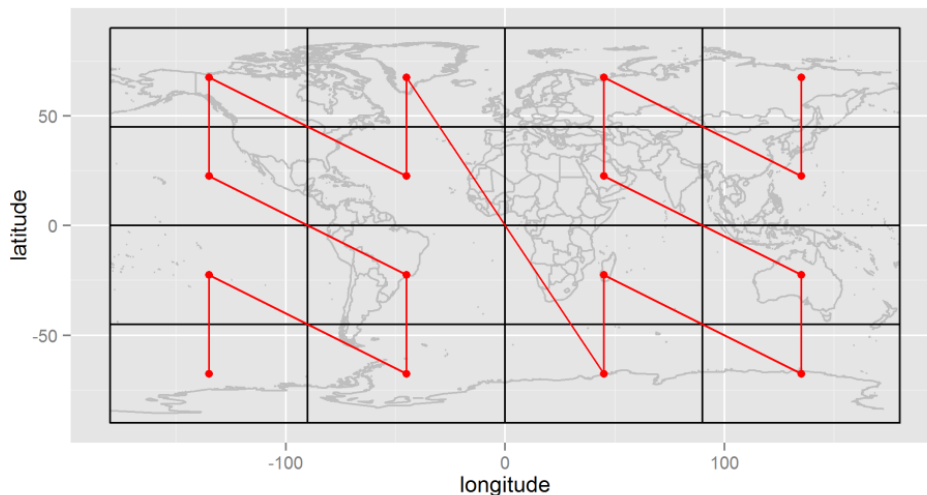


Figure 3. Trajectoire spatiale en Z sur une planisphère

GEOHASH réalise la même opération avec une granularité plus fine de 32 divisions à chaque étape.

Principal défaut : la proximité dans l'index ne correspond pas toujours à celui d'une recherche par proximité géographique car une zone de recherche peut être à cheval sur plusieurs segments

possédant des en-têtes de clef totalement différents. En revanche la recherche par proximité fonctionne particulièrement bien lorsqu'elle se déroule intégralement dans le segment de plus fine granularité. Le choix de la granularité la plus adaptée est donc primordial pour chaque recherche par proximité.

MongoDB, **Elasticsearch**, **Hbase** et **Accumulo** (entre autres) utilisent GEOHASH pour les indexes geo-spatiaux. Certains systèmes propriétaires utilisent des variantes supposées plus efficaces sur les points exposés.

III.7. Pour plus de précision : indexation par arbre R

L'arbre R est une variante des arbres B dédiée à l'indexation multi-dimensionnelle. C'est la solution la plus traditionnelle en matière d'indexation spatiale, mais pas encore la plus employée sur des systèmes de données massives, notamment par sa conception originelle non distribuée. Il semble cependant que des variantes scalables aient été définies, comme SD-RTREE [RIG2006].

L'idée principale par rapport au GEOHASH consiste à ne pas segmenter arbitrairement la surface selon un découpage choisi empiriquement, mais au contraire d'examiner les données pour former des zones spatiales de tailles variables mais regroupant les objets les plus proches. Ces zones sont ensuite organisées hiérarchiquement de façon similaire à un arbre B ou B+. Pour simplifier les calculs, ces zones restent rectangulaires (d'où le « R » du nom).

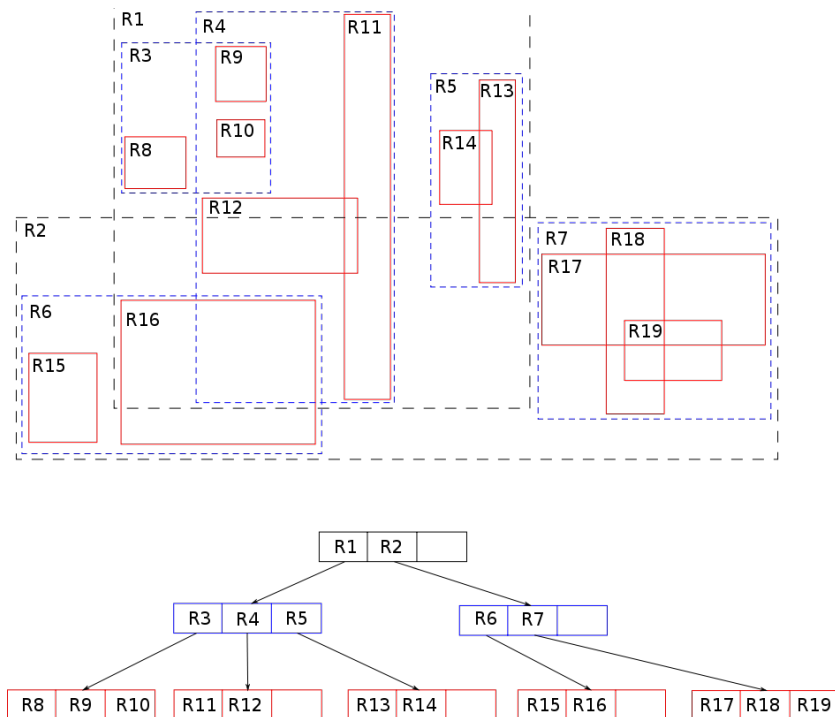


Figure 4. Exemple de segmentation de plan par arbre-R

L'avantage immédiat par rapport à GEOHASH, est que deux zones spatialement proches ne sont jamais éloignées dans l'arbre : les recherches de proximité sont donc toujours efficaces !

L'autre avantage de cet algorithme est qu'on peut généraliser la segmentation à un nombre quelconque de dimensions, notamment à 3 ou 4 pour ce qui concerne les données spatio-temporelles. Évidemment, plus le nombre de dimensions est important, plus la segmentation est problématique en temps de calcul, car on se ramène à un problème classique de classification. Un exemple de segmentation 3D par arbre R est visible en figure 5.

Le désavantage majeur par rapport au simple calcul d'un GEOHASH est que l'insertion dans

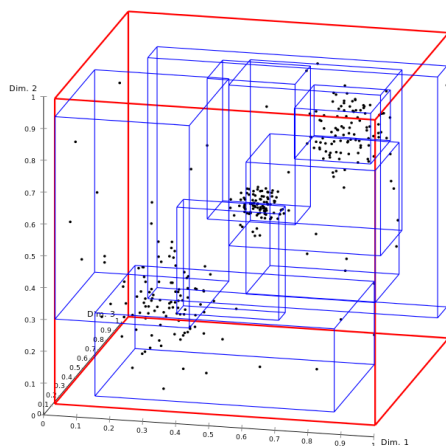


Figure 5. Exemple de segmentation tridimensionnelle par arbre-R

l'arbre est largement plus compliquée, même si la complexité de cette opération reste idéale (ordre $O(n)$). De plus, les zones rectangulaires sont susceptibles de se recouvrir, ce qui peut ralentir une recherche à granularité fine. Il existe cependant plusieurs variantes (dont R^+ et R^*), spécialisées selon les cas d'utilisation. Cette famille d'algorithmes semble faire actuellement l'objet d'une recherche continue.

Durant le déroulement de ce projet, nous avons pas trouvé d'implémentation libre de l'arbre R qui soit livrée « clef en main » avec un système de stockage distribué et scalable. Cependant, de nombreux systèmes commerciaux mettent en avant leur capacité à distribuer ce type d'index. Dans notre cas, d'autres moyens sont heureusement proposés par les outils disponibles librement.

III.8. Solution alternative : indexation des GEOHASH avec Lucene/ElasticSearch

Si l'algorithme GEOHASH est parfait pour référencer des localisations, il ne permet pas par lui-même d'indexer des documents géolocalisés, contrairement à l'arbre R qui contient les données (ou des références sur les données).

Lucene est une bibliothèque d'indexation textuelle et ne supporte donc nativement aucune des méthodes géospatiales. Par contre, rien n'empêche d'indexer textuellement les chemins menant à chaque donnée d'un arbre. On détermine quels éléments de l'arbre sont couverts par chaque polygone (zone rectangulaire aussi bien en R-tree qu'en GEOHASH). Sur le principe du GEOHASH, on calcule pour chaque zone couvrante (rectangle) une clef décrivant le chemin de la racine à la feuille. Cette clef est une simple chaîne binaire ou textuelle, ce que Lucene est précisément capable de gérer comme un « terme » !

A partir de là, tout se déroule comme d'ordinaire avec **Elasticsearch** : chaque zone géographique est considérée comme un polygone contenant un certain nombre de nœuds de l'arbre. Cette détermination se fait rapidement grâce à la structure implicite du GEOHASH. Lors d'une recherche de proximité, il suffit donc de retenir les éléments de la collection contenant au moins un des valeurs de GEOHASH retenues.

Le système est simple à mettre en place et immédiatement distribuable grâce à **Elasticsearch**. Il a cependant 2 défauts :

- la précision de toute recherche dépend de la longueur des GEOHASH préalablement indexés.
- La taille de l'index augmente exponentiellement selon la longueur choisie pour les GEOHASH.

Il importe donc de faire ce choix de longueur de manière adaptée aux besoins en précision et de l'espace disponible pour stocker l'index.

De plus, cette solution ne tient pas compte de la dimension temporelle, il est donc indispensable de continuer à utiliser un index secondaire pour la navigation sur cet axe.

Partie IV

Pour aller plus loin : vers un modèle de données hybride

Ce chapitre extrapole sur le prolongement du projet personnel en étudiant une des tendances actuelles en matières de systèmes NoSQL, celle allant vers la description de modèles de données de plus en plus riches et souples. Nous présenterons en particulier le système **OrientDB**, un des systèmes actuels les plus évolués dans ce domaine particulier et répondant bien à nos besoins particuliers.

IV.1.1. *Rappel des besoins*

Ainsi que nous l'avons mentionné en partie II Expression des besoins, nous savons que dans le cadre de la poursuite de ce projet en fouille de données, nous aurons également besoin de construire une base de faits à partir des données brutes (les données brutes étant celles que nous avons archivées dans **MongoDB**).

Cette base de faits doit permettre d'enregistrer les liaisons ou relations entre les différentes observations (par exemple : la proximité géographique entre 2 centres d'activité sismique, associés également par la distance temporelle, mais encore par leurs éventuels points communs en termes d'intensité et de catégorisation des manifestations). On peut représenter naturellement de telles données sous forme de graphes valués. Or, l'archivage de ce type de données est reconnu peu efficace dans un SGBD relationnel, mais aussi dans un système à base de collections comme **MongoDB**, car ici la majorité des données sont des relations entre les enregistrements, et non des valeurs littérales. Les relations étant dans ces 2 cas stockées sous forme de références d'enregistrements, les procédures de recherche prennent énormément de temps car il faut plusieurs parcours de la même table ou de la même collection pour obtenir l'ensemble des nœuds d'un graphe. Les « jointures » multiples de table deviennent coûteuses.

Quand à **MongoDB**, il est incapable de réaliser lui même l'équivalent de la jointure relationnelle : toutes les relations doivent être suivies à l'aide de boucles de parcours, ce qui est loin de fournir des requêtes faciles à lire et à modifier.

Pour finir, mentionnons que **MongoDB** est disponible en open-source sous licence Apache.

IV.1.2. Rappel des principaux paradigmes de modélisation de données

Toute application informatique de manipulation de données décrit une représentation interne de ces données qu'on appelle *modèle de données*, chaque « modèle » étant la description du schéma des données. Il existe cependant plusieurs façons de décrire un schéma pour les les mêmes données ; on nomme chacune de ces façons « PARADIGME », ce qui signifie, dans ce contexte, « *modèle de description des modèles* ».

Outre le bien connu modèle relationnel, un certain nombre de paradigmes a été évoqué en cours de **NFE204**, à défaut d'être exploités systématiquement. En voici un rapide rappel, dont une représentation détaillée est visible ici en figure 6. A cette catégorisation en termes de représentation des données à un instant fixé, on ajoutera ensuite d'autre sous-catégorisations sur des critères orthogonaux.

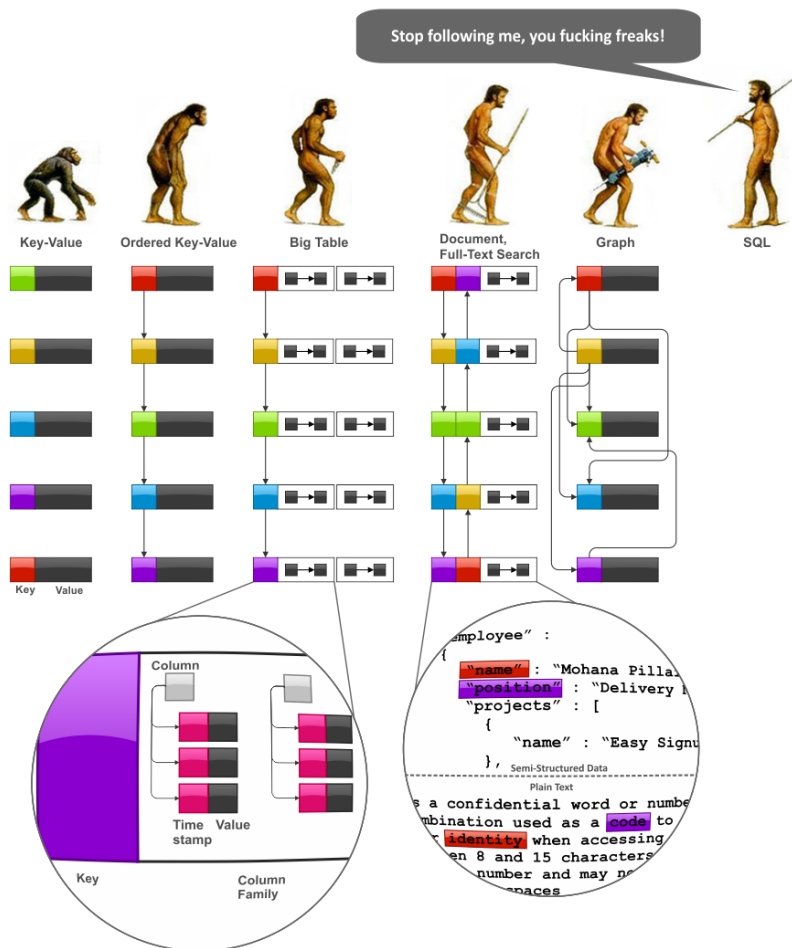


Figure 6. Les principaux paradigmes de modélisation de données
 (Très belle synthèse outrageusement empruntée au blog Highly Scalable [www2])

- les systèmes de **paires clef-valeur**, ordonnées ou non, qui forment naturellement un index unidimensionnel vers les enregistrements (cas le plus simple imaginable). Rien n'empêche, naturellement, d'ajouter à volonté des indexes secondaires en plus de cette CLEF NATURELLE. Une éventuelle relation d'ordre permet, en complément, un accès séquentiel aux enregistrements.
 - les systèmes à « **colonnes variables** », où la clef naturelle de chaque enregistrement est , cette fois, composite et tripartite (parties traditionnellement nommées « ligne », « colonne » et « temps »). On peut donc parler d'indexation naturelle « en cube ». Le modèle général d'indexation à n composantes (n étant une valeur fixée définitivement) est dit « en HYPERCUBE ».
 - les systèmes dits « **orientés documents** », où la multiplicité des champs et l'imbrication illimitée de ceux-ci fait qu'il n'existe aucune clef naturelle. Seul le programmeur est à même de définir des indexes en fonction des données utilisées. En l'absence de ceux-ci, l'accès de base aux données est donc séquentiel car il ne peut se faire que par chaînage (ce que la plupart des systèmes évite dans la pratique en générant automatiquement au moins une CLEF ARTIFICIELLE).
 - Enfin, les systèmes « **orientés graphes** », où, non seulement il n'existe pas de clef naturelle sur les enregistrements, mais il n'existe même plus d'ordre de chaînage naturel entre ceux-ci : toutes les relations entre enregistrements étant cette fois définies par les données elles-mêmes.
- On remarquera ici que l'accès aux différents attributs d'un enregistrement n'est pas spécifiquement défini par ce paradigme, ce qui laisse plusieurs possibilités sur cet aspect.

IV.1.3. Et maintenant : pourquoi se limiter à un seul paradigme ?

Chacun des paradigmes que nous venons de décrire est connu depuis des décennies dans le domaine de l'informatique, à l'exception toutefois du modèle dit « à colonnes variables » qui est en fait une spécialisation industrielle récente (Google **BigTable**, 2005) d'un modèle d'indexation en HYPERCUBE.

Malgré l'ancienneté de ces modèles, il est évident que seul l'essor industriel continu d'internet depuis les années 2000 a permis leur vulgarisation et leur mise en application à travers tout un panel de nouveaux outils.

Mais, bien que ces modèles théoriques permettent une classification bien pratique de l'aspect modélisation de chaque outil NoSQL, ils n'en restent pas moins théoriques. L'essor du mouvement NoSQL a en effet encouragé les industriels à l'hyper-personnalisation des outils en fonction des besoins d'exploitation, ce qui a notamment poussé à l'hyper-personnalisation des modèles de données. La personnalisation dans ce domaine a donc abouti à des systèmes « hybrides », se partageant allègrement entre plusieurs paradigmes de modélisation des données.

Les principaux outils de ce genre disponibles sur le marché sont énumérés en tableau 2.

Système	Paradigmes représentés	Première apparition	Multi-paradigme depuis
Virtuoso	relationnel étendu objet documents XML documents plein texte graphes RDF tables à colonnes variables	1998	2008
MarkLogic	documents JSON documents XML graphes RDF	2001	2013
OrientDB	documents plein texte documents XML graphes	2010	2010
Amazon DynamoDB	clef-valeur documents plein texte documents XML documents JSON	2012	2014
ArangoDB	clef-valeur documents graphes	2012	2013
Sqrrl	documents graphes clef-valeur table à colonnes variables	2012	2012
FoundationDB	clef-valeur documents relationnel	2013	2013
Amisa Server	documents graphes relationnel	2013	2013

Tableau 1. Principaux outils à modélisation hybride disponibles actuellement

On constate deux phénomènes distincts dans l'apparition de ces produits hybrides. D'une part, nous avons quelque systèmes anciens tels que **Virtuoso** et **Marklogic** qui au fil des évolutions ont fini par adopter plusieurs paradigmes. D'autre part, nous avons une collection de systèmes récents, ouvertement d'appartenance NoSQL, et conçus dès le départ pour une modélisation de données sur plusieurs paradigmes. Ce dernier point n'empêchant pas ces systèmes récents d'avoir chacun déjà connu plusieurs évolutions sur ce sujet.

On remarque également que certaines bases supportent séparément différents formats de « documents », plus ou moins structurés : ce que je mentionne dans ce cas est un support interne de fonctionnalités structurées spécifiques à ces formats. Dans les cas contraires, le simple support de documents « plein texte » oblige à traiter séparément les données extraites de la base avec une API de son choix, ce qui est obligatoirement moins efficace.

Inversement, on constate que certains systèmes dont l'évolution a répondu à des besoins très spécifique comme **Marklogic** ne supportent que les documents structurés selon certains formats et pas les documents « plein texte ».

Cette restriction des API est encore plus importante pour les graphes : la structure de graphe la plus répandue est RDF, destinée initialement au web sémantique. Peu de bases proposent à la fois des fonctions pour les graphes génériques et le RDF. Il semble que le format de description de graphes génériques actuellement le plus reconnu est **BluePrint**.

IV.2. Manipulation de données orientées graphes : quels langages ?

IV.2.1.1. Le langage SPARQL

C'est un langage d'interrogation par sélection et projection mettant en avant la notion de relation entre éléments. Il a à l'origine été créé pour exploiter le format RDF destiné à la description du web sémantique, mais sa syntaxe peut être adaptée sur la majorité des graphes mono-valués. La seule contrainte est que toute relation est représentée par un triplet (origine, destination, valeur) pour s'adapter au format (sujet, prédicat, objet) de RDF.

Le principe de jointure du SQL est remplacé par celui de « traversée de relation » de proche en proche, c'est à dire de nœud en nœud. L'exemple ci-dessous permet d'interroger un réseau de relations sociales pour obtenir les noms de mariage de toutes les descendantes de monarchies européennes.

```
PREFIX ged: <http://www.daml.org/2001/01/gedcom/gedcom#>
SELECT ?name ?marriedOn
FROM <http://www.daml.org/2001/01/gedcom/royal92.daml>
WHERE
{
  ?royal ged:title "Princess".
  ?royal ged:name ?name.
  ?royal ged:spouseIn ?family.
  ?family ged:marriage ?marriage
  ?marriage ged:date ?marriedOn.
}
ORDER BY ASC [?name]
```

Le fonctionnement sans schéma de SPARQL et syntaxe ouverte lui permet apparemment de fonctionner sur tous les types de modèles de données, depuis les magasins de clef-valeur aux bases de graphes, en passant par les structures de tables avec associations. Il partage également avec NoSQL l'approche « bottom-up » : on peut construire ultérieurement une requête complexe à partir de plusieurs requêtes simples. Cette caractéristique le rend facile d'accès pour l'apprentissage.

IV.3. Solution 1 : AllegroGraph

AllegroGraph peut s'interfacer avec une base **MongoDB** pour l'interroger en langage SPARQL, simplement en lui ajoutant un index spécialisé pour des données de type triplet au format RDF. Mais les performances de cet assemblage sont réputées médiocre en cas mise à jour fréquente des données (ainsi que nous le souhaitons pour répondre au besoin n°2 de II Expression des besoins) car il faut alors recalculer régulièrement cet index supplémentaire, ce qui est coûteux. **[www4]** De plus, il n'est pas possible d'ajouter des propriétés valuées sur une relation au format RDF.

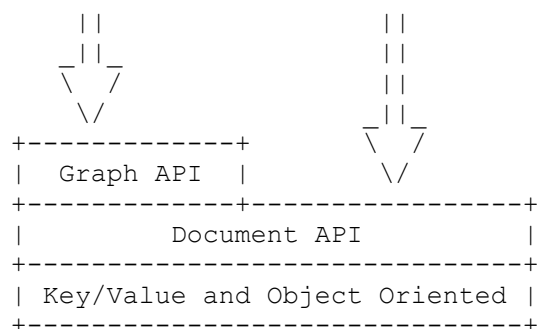
IV.4. Solution 2 : OrientDB

IV.4.1. Caractéristiques retenues

- Base de graphes génériques avec chaque nœud sous forme de document semi-structuré JSON, auxquels se rajoute encore la notion d'héritage objet par prototypage ;
- Il est possible de spécifier un schéma pour les documents, pour les graphes, pour les deux ou pas du tout ! Donc beaucoup de souplesse pour garantir l'intégrité des données !
- Isolation des transactions : supporte la totalité des propriétés **ACID** (Atomicité, Cohérence, Isolation, Durabilité), alors que le système est totalement distribué ;
- Langage d'interrogation par défaut proche de SQL, mais étendu pour permettre la descente de relations et la représentation de quelques concepts objets comme l'héritage ;
- Sa conformité à la norme **BluePrint** (description de graphes) le rend également exploitable avec le langage SPARQL, à l'aide du framework **Sesame**.
- Partitionnement : Alors que des systèmes plus anciens comme **Neo4j** sont exclusivement centralisés, **OrientDB** offre le partitionnement complet par *sharding*.
- Réplication : type master-slave ;
- Impossible de déclarer un traitement en MapReduce, mais les principales agrégations connues sont réalisées automatiquement de façon distribuée par le langage de requête ;
- Interface RESTFUL pour une API universelle et une intégration facile au web ;
- Sous licence Apache v2, donc répondant à l'expression des besoins sur ce point.

IV.3.2. Exploitation

Les données de OrientDB sont structurées en 3 niveaux encapsulés, dont 2 sont directement accessibles par une API java. Chaque élément de la base est donc fondamentalement un document semi-structuré qui peut, à son tour, être considéré comme le nœud d'un graphe, mais pas obligatoirement.



L'API en Java permet un accès plus précis aux méthodes que le langage ORIENT SQL (qui permet par contre une plus forte abstraction dans la manipulation). Dans tous les cas, nous avons ici un choix utile pour nous adapter selon nos besoins de stockage.

On voit qu'il serait fonctionnellement possible de remplacer également **MongoDB** sur le [besoin n°1](#) (stockage de données brutes) , en utilisant cette fois **OrientDB** comme un simple *document-store*. Ce point reste cependant à confirmer en terme de performances comparées par rapport à **MongoDB**. De plus, il n'existe actuellement pas d'équivalent du MapReduce dans **OrientDB**, bien que les agrégations soient automatiquement distribuées. C'est à l'occasion du projet **RCP216** que nous devrions pouvoir déterminer si les opérations à effectuer sur les données brutes justifient l'emploi de la distribution de traitements personnalisés. Dans l'intervalle, nous continuerons à utiliser **MongoDB** pour le besoin n°1.

Nous concluons donc que, à ce stade du projet, l'utilisation simultanée de plusieurs systèmes NoSQL reste le meilleur moyen d'en comparer l'efficacité et les performances pour chaque besoin spécifique.

Principales références bibliographiques

- [RIG2006] DU MOUZA C., LITWIN W., RIGAUX P., *SD-Rtree: A Scalable Distributed Rtree*, 2006.
- [www1] *B+Trees – How SQL Server Indexes are Stored on Disk*, <http://sqlity.net/en/2445/b-plus-tree/>
- [www2] *Highly Scalable Blog, NoSQL Data Modeling Techniques, Mars 2012*
<https://highlyscalable.wordpress.com/2012/03/01/nosql-data-modeling-techniques/>
- [www3] *R-tree*, wikipedia http://en.wikipedia.org/wiki/R_tree
- [www4] *AllegroGraph Triple Indices*, <http://franz.com/agraph/support/documentation/current/triple-index.html>
- [www4] OrientDB Documentation, <http://www.orienttechnologies.com/docs/last/>